

constant computation framework discussed in Example 10.47; the relevant flow graph is shown in Fig. 10.63. The mappings μ and ν coming out of B_2 and B_4 are the ones from Example 10.47. Mapping ρ , entering B_5 , is $\mu \wedge \nu$, and σ is the mapping coming out of B_5 , setting a to *nonconst*, even though every real path (and every apparent path), computes $a = 5$ after B_5 .

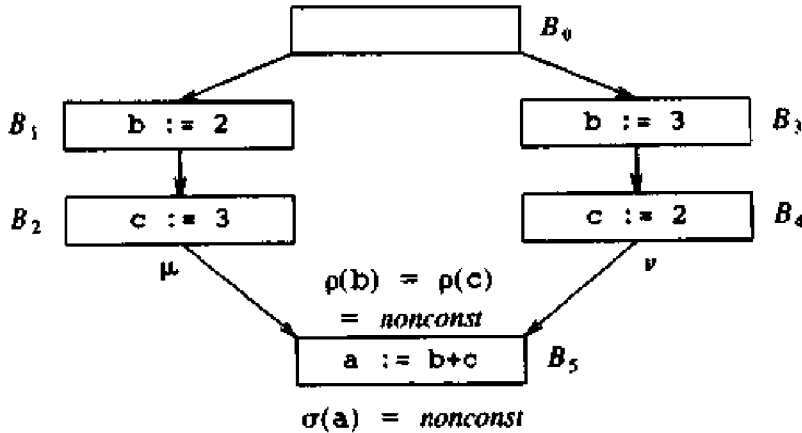


Fig. 10.63. Example of solution less than *mop* solution.

The problem, intuitively, is that Algorithm 10.18, dealing with a nondistributive framework, behaves as if some sequences of nodes that are not even apparent paths (paths in the flow graph), were real paths. Thus, in Fig. 10.63, the algorithm behaves as if paths like $B_0 \rightarrow B_1 \rightarrow B_4 \rightarrow B_5$ or $B_0 \rightarrow B_3 \rightarrow B_2 \rightarrow B_5$ were real paths, setting b and c to a combination of values that do not sum to five. □

Convergence of Algorithm 10.18

There are various ways that we could prove Algorithm 10.18 converges for a particular framework. Probably the most common case is where only acyclic paths are needed, i.e., we can show that the meet over acyclic paths is the same as the *mop* solution, over all paths. If that is the case, not only does the algorithm converge, it will normally do so very rapidly, in two more passes than the depth of the flow graph, as we discussed in Section 10.10.

On the other hand, frameworks like our constant computation example require more than acyclic paths be considered. For example, Fig. 10.64 shows a simple flow graph where we have to consider the path $B_1 \rightarrow B_2 \rightarrow B_2 \rightarrow B_3$ to realize that x does not have a constant value on entering B_3 .

However, for constant computations, we can reason that Algorithm 10.18 converges as follows. First of all, it is easy to show for an arbitrary monotone framework that $in[B]$ and $out[B]$, for any block B , form a nonincreasing sequence, in the sense that the new value for one of these variables is always $\leq$ the old value. If we recall Fig. 10.60, the lattice diagram for the values of mappings applied to variables, we realize that for any variable, the value of

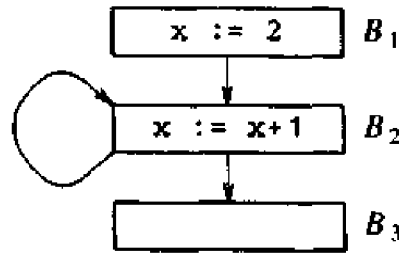


Fig. 10.64. Flow graph requiring cyclic path be included in *mop*.

$in[B]$ or $out[B]$ can only drop twice, once from *undef* to a constant, and once from that constant to *nonconst*.

Suppose there are n nodes and v variables. Then on every iteration of the while-loop in Fig. 10.62, at least one variable must have its value drop in some $out[B]$, or the algorithm converges, and even infinite iteration of the while-loop will not change the values of the *in*'s or *out*'s. Thus, the number of iterations is limited to $2nv$; if that number of changes occurs, then every variable must have reached *nonconst* at every block of the flow graph.

Fixing the Initialization

In some data-flow problems, there is a discrepancy between what Algorithm 10.18 gives us as a solution and what we intuitively want. Recall that for available expressions, $\wedge$ is intersection, so T must be the set of all expressions. Since Algorithm 10.18 initially assumes $in[B]$ is T for each block B , including the initial node, the *mop* solution produced by Algorithm 10.18 is actually the set of expressions that, assuming they are available at the initial node (which they are not), would be available on entrance to block B .

The difference, of course, is that there might be paths from the initial node to B along which an expression $x+y$ is neither generated nor killed. Algorithm 10.18 would say $x+y$ is available, when in fact it is not, because no variable along that path can be found to hold its value. The fixup is simple. We can either modify Algorithm 10.18 so that for the available expression framework, $in[B_0]$ is set and held equal to the empty set, or we can modify the flow graph by introducing a dummy initial node, a predecessor of the real initial node, that kills every expression.

10.12 ESTIMATION OF TYPES

We now come to a data-flow problem that is more challenging than the frameworks of the previous section. Various languages, ranging from APL to SETL to the many dialects of Lisp, do not require that the types of variables be declared, and even permit the same variable to hold values of different types at different times. Serious attempts to compile such languages into efficient code have used data-flow analysis to infer the types of variables, since

code to, say, add two integers, is far more efficient than a call to a general routine that can add two objects of a variety of types (e.g., integer, real, vector).

Our first guess might be that computing types of variables is something like computing reaching definitions. We can associate a set of possible types with each variable at each point. The confluence operator is union on sets of types, since if variable x has set S_1 of possible types on one path and set S_2 on another, then x has any of the types in $S_1 \cup S_2$ after the confluence of the paths. As control passes through a statement, we may be able to make some inferences about the types of variables based on the operators in the statement, the possible types of their operands, and the types they produce as results. Example 6.6, which dealt with an operator that could multiply both integers and complex numbers, was an example of this sort of inference.

Unfortunately, there are at least two problems with this approach.

1. The set of possible types for a variable may be infinite.
2. Type determination usually requires both forward and backward propagation of information to obtain precise estimates of possible types. Thus, even the framework of Section 10.11 is not general enough to do justice to the problem.

Before considering point (1), let us examine some of the kinds of inferences about types that can be made in familiar languages.

Example 10.50. Consider the statements

```
i := a[j]
k := a[i]
```

Suppose at first we do not know anything about the types of the variables a , i , j , and k . However, let us suppose that the array accessing operator $[]$ requires an integer argument. By examining the first statement we may infer that j is an integer at that point, and a is an array of elements of some type. Then, the second statement tells us that i is an integer.

Now, we may propagate inferences backward. If i was computed to be an integer in the first statement, then the type of expression $a[i]$ must be integer, which means that a must be an array of integers. We can then reason forward again to discover that the value assigned to k by the second statement must also be an integer. Notice it is impossible to discover that the elements of a are integers by reasoning only forward or only backward. $\square$

Dealing with Infinite Type Sets

There are numerous examples of pathological cases where the set of possible types for a variable really is infinite. For example, SETL allows a statement like

```
x := { x }
```

to be executed inside a loop. If we start out knowing only that x could be an integer, then after we consider one iteration of the loop, we realize x could be either an integer or a set of integers. After considering a second iteration we find that x could also be a set of sets of integers, and so on.

A similar problem could occur in a typeless version of a conventional language like C, where the statement

$$x = \&x$$

with the initial possibility that x is an integer leads us to discover that x can have any type of the form

pointer to pointer to $\dots$ pointer to integer

The traditional way to deal with such problems is to reduce the set of possible types to a finite number. The general idea is to group the infinite number of possible types into a finite number of classes, generally keeping the simpler types by themselves, and grouping the most complicated, and hopefully the rarest, types into large classes. When we do so, we have to exercise judgment as to how we make inferences about the interaction between types and operators. The following example suggests what can be done.

Example 10.51. Let us continue the example of Chapter 6, where we used the operator $\rightarrow$ as a type constructor for functions. Here, our set of types will include the basic type *int*, and all types of the form $\tau \rightarrow \sigma$, representing the type of a function with domain type τ and range type σ , where τ and σ are types in our set. Thus, our set of types is infinite, including such types as

$$(int \rightarrow int) \rightarrow ((int \rightarrow int) \rightarrow int)$$

To reduce this set to a finite number of classes, we shall restrict a type expression to have only one function type constructor $\rightarrow$, by replacing subexpressions in a type expression containing at least one occurrence of $\rightarrow$ by the name *func*. Thus, there are five different types:

$$\begin{aligned} &int \\ &int \rightarrow int \\ &int \rightarrow func \\ &func \rightarrow int \\ &func \rightarrow func \end{aligned}$$

We shall represent sets of types as bit vectors of length five, with the positions corresponding to the five types in the order listed above. Thus, 01111 represents the type for any function application, i.e., for anything but *int*. Note that this is in a sense the type of *func*, since a *func* may not be an integer.

The basic assignment statement for our model is

$$x := f(y)$$

Knowing the possible types of f and y , we can determine the possible types

of x by looking up the type in the table of Fig. 10.65. If f can be of any type in the set S_1 and y of any type in set S_2 , we take each pair τ in S_1 and σ in S_2 and look up the entry in the row for τ and the column for σ , which we call $\tau(\sigma)$. We then take the union of the results of all these lookups to get the set of possible types for x .

τ	σ				
	<i>int</i>	<i>int</i> → <i>int</i>	<i>int</i> → <i>func</i>	<i>func</i> → <i>int</i>	<i>func</i> → <i>func</i>
<i>int</i>	00000	00000	00000	00000	00000
<i>int</i> → <i>int</i>	10000	00000	00000	00000	00000
<i>int</i> → <i>func</i>	01111	00000	00000	00000	00000
<i>func</i> → <i>int</i>	00000	10000	10000	10000	10000
<i>func</i> → <i>func</i>	00000	01111	01111	01111	01111

Fig. 10.65. The value of $\tau(\sigma)$.

For example, if $\tau = \textit{int} \rightarrow \textit{func}$ and $\sigma = \textit{int}$, then $\tau(\sigma) = 01111$. That is, the result of applying a mapping of type $\textit{int} \rightarrow \textit{func}$ to an $\textit{int}$ is a $\textit{func}$, which means a mapping of any of the four types besides $\textit{int}$. We cannot tell which because our smearing of an infinite number of types into five classes precludes our knowing.

For a second example, let τ be as before and $\sigma = \textit{int} \rightarrow \textit{int}$. Then $\tau(\sigma) = 00000$ because the domain type of τ is definitely unequal to the type σ , and therefore the mapping is inapplicable. □

A Simple Type System

To illustrate the ideas behind our type-inference algorithms, we introduce a simple type system and language based on Example 10.51. The types are the five illustrated in that example. The statements of our language are of three kinds.

- 1. **read x .** A value of x is read from the input, and presumably, nothing is known about its type.
- 2. **$x := f(y)$.** The value of x is set to be that obtained by applying function f to value y . What is known about the type of x after the assignment is summarized in Fig. 10.65.
- 3. **use x as τ .** When we go through such a statement, we may assume that the program is correct, and therefore, the type of x can only be τ both before and after the statement. The value and type of x are not affected by the statement.

We infer types by performing a data-flow analysis on a flow graph of a program consisting of statements of these three types. For simplicity, we assume

that all blocks consist of a single statement. The values of *in* and *out* for blocks are mappings from variables to sets of the five types from Example 10.51.

Initially, each *in* and *out* maps every variable to the set of all five types. As we propagate information, we reduce the set of types associated with certain variables at certain points, until at some time we cannot reduce any of these sets any further. The resulting sets will be assumed to indicate the possible types of each variable at each point. That assumption is conservative, since a type is eliminated only if we can prove (given that the program is correct) that the type is impossible. Normally, we expect to take advantage of the fact that some types are impossible, not that they are possible, so "too large" is the safe direction for errors.

We use two schemes to modify the *in*'s and *out*'s: a "forward" scheme and a "backward" scheme. The forward scheme uses the statement in a block *B* and the value of *in*[*B*] to restrict *out*[*B*],¹⁶ and the backward scheme does the opposite. In each scheme, the confluence operator is "variable-wise union," in the sense that the confluence of two mappings α and β is that mapping γ such that for all variables *x*,

$$\gamma[x] = \alpha[x] \cup \beta[x]$$

The Forward Scheme

Let us suppose we have a block *B* with *in*[*B*] the mapping μ and *out*[*B*] the mapping ν . The forward scheme lets us restrict ν . The rules for restricting ν depend upon what instruction is found in block *B*, naturally.

1. If the statement is `read x`, then any type could be read. If we already know something about the type of *x* after the read, we must not forget it during this forward pass, so we simply do not change $\nu(x)$ on the forward pass. For all other variables *y*, we set

$$\nu(y) := \nu(y) \cap \mu(y)$$

2. Now suppose the statement is `use x as  $\tau$` . After this statement, τ is the only possible type for *x*. If we already know that type τ is impossible for *x*, then there is no possible type for *x* after the statement. These observations can be summarized by:

¹⁶ It is worth noting that in traditional forward data-flow schemes, we did not restrict *out*, but rather computed it anew from *in* each time. We could do this because *in*'s and *out*'s always changed in one direction, either always growing or always shrinking. However, in a problem like type inference, where we alternately perform forward and backward passes, we may have a situation where the backward pass has left *out* much smaller than we can justify by applying the forward rules to *in*. Thus, we must not accidentally raise *out* on the forward pass, only to have it lowered again (but perhaps not as far) on the backward pass. A similar remark applies to the backward pass: we must restrict *in*, not recompute it.

$$\begin{aligned}v(x) &:= v(x) \cap \{\tau\} \\v(y) &:= v(y) \cap \mu(y) \text{ for } y \neq x\end{aligned}$$

3. Now, consider the case that the statement is $x := f(y)$. The only possible types for x after the statement are those that
 - i) are possible according to the present value of v , and
 - ii) are the result of applying a mapping of some type τ to type σ , and τ and σ are types that f and y , respectively, could have before the statement is executed.

Formally,

$$v(x) := v(x) \cap \{\rho \mid \rho = \tau(\sigma), \tau \text{ is in } \mu(f), \text{ and } \sigma \text{ is in } \mu(y)\}$$

We may also make some inferences about the types of f and y , since on the assumption of program correctness, f cannot have a type that doesn't apply to some type of y , and y cannot have a type that cannot serve as the argument type for some possible type of f . That is, if $f \neq x$ then

$$v(f) := v(f) \cap \{\tau \text{ in } \mu(f) \mid \text{for some } \sigma \text{ in } \mu(y), \tau(\sigma) \neq \emptyset\}$$

if $y \neq x$ then

$$v(y) := v(y) \cap \{\sigma \text{ in } \mu(y) \mid \text{for some } \tau \text{ in } \mu(f), \tau(\sigma) \neq \emptyset\}$$

for all other z ,

$$v(z) := v(z) \cap \mu(z)$$

The Backward Scheme

Now let us consider how, in a backward pass, we can restrict μ based on what v tells us and what the statement tells us.

1. If the statement is `read x`, it is easy to see that no new inferences about impossible types before the statement can be made, so $\mu(x)$ does not change. However, for all $y \neq x$, we may propagate information backward by setting $\mu(y) := \mu(y) \cap v(y)$.
2. If we have the statement `use x as  $\tau$` , then we can make the same sort of inference we made in the forward direction; x can only have type τ before the statement, and any other variable's types are those that are believed possible both before and after the statement. That is:

$$\begin{aligned}\mu(x) &:= \mu(x) \cap \{\tau\} \\ \mu(y) &:= \mu(y) \cap v(y) \text{ for } y \neq x\end{aligned}$$

3. As before, the most complex case is a statement of the form $x := f(y)$. To begin, nothing new can be inferred about x before the statement, unless x happens to be one of f or y . Thus, $\mu(x)$ is not changed except by the following rules concerning f and y . Next, note that as in the forward rules, we may make inferences from the fact that the types of f and y must be compatible before the statement. However, if $f \neq x$, we may

also restrict $\mu(f)$ to the types in $\nu(f)$, and an analogous statement holds about y . On the other hand, if $f = x$, then the types of f after the statement are unrelated to the types of f before, so no such restriction is permitted. Again, an analogous statement holds if $y = x$. It is useful to define a special mapping, just for f and y , to reflect this decision. Thus, we define:

if $f = x$ then $\mu_1(f) := \mu(f)$ else $\mu_1(f) := \mu(f) \cap \nu(f)$
 if $y = x$ then $\mu_1(y) := \mu(y)$ else $\mu_1(y) := \mu(y) \cap \nu(y)$

Now, we can restrict f and y to those types that are compatible with the other's set of types. At the same time, we can restrict the types of f and y based on the fact that they must not only be compatible, but must yield a type that ν says x may have. Thus, we define:

$\mu(f) := \{\tau \text{ in } \mu_1(f) \mid \text{for some } \sigma \text{ in } \mu_1(y), \tau(\sigma) \cap \nu(x) \neq \emptyset\}$
 $\mu(y) := \{\sigma \text{ in } \mu_1(y) \mid \text{for some } \tau \text{ in } \mu_1(f), \tau(\sigma) \cap \nu(x) \neq \emptyset\}$
 $\mu(z) := \mu(z) \cap \nu(z) \text{ for } z \text{ not equal to } x, y, \text{ or } f$

Before proceeding to the type determination algorithm, let us recall from our discussion of reaching definitions in Section 10.5 that if we start with the false assumption that some definition d is available at some point in a loop, we may erroneously propagate that fact around the loop, leaving us with a set of reaching definitions that is larger than necessary. A similar problem can occur in type determination, where the assumption that a variable can have a certain type "proves" itself as we go around a loop. Therefore, we shall introduce a 33rd value, in addition to the 32 sets of types from Example 10.51, that a mapping μ may assign to a variable, the value *undef*. This use of *undef* is similar to its use in the constant propagation framework of the previous section.

During confluence, the value *undef* yields to any other value, i.e., it acts like the type 00000. On the other hand, when intersecting sets of types, e.g., computing $\mu(x) \cap \nu(x)$, the value *undef* also yields to any other set of types; that is, it functions like the type 11111. Thus, for example, when we read a value of a variable x , the fact that the "type" of x was thought to be *undef* after the read is overruled, and the type of x becomes 11111.

Algorithm 10.19.

Input. A flow graph whose blocks are single statements of the three types (read, assignment, and use-as) mentioned above.

Output. A set of types for each variable at each point. The set is conservative, in the sense that any real computation must lead to a type in the set.

Method. We compute a mapping $in[B]$ and a mapping $out[B]$ for each block B . Each mapping sends the program's variables to sets of types in the type system introduced in Example 10.51. Initially, all mappings send each variable to *undef*.

We then make alternate forward and backward passes through the flow graph, until consecutive forward and backward passes both fail to make any changes. The forward pass is performed by:

```

for each block  $B$  in depth-first order do begin
     $in[B] := \bigcup_{pred. P \text{ of } B} out[P];$ 
     $out[B] :=$  function of  $in[B]$  and  $out[B]$  as defined above
end

```

The backward pass is:

```

for each block  $B$  in reverse depth-first order do begin
     $out[B] := \bigcup_{succ. S \text{ of } B} in[S];$ 
     $in[B] :=$  function of  $in[B]$  and  $out[B]$  as defined above
end

```

□

Example 10.52. Consider the simple straight-line program shown in Fig. 10.66. We are interested in four mappings, which we designate μ_1 through μ_4 . Each μ_i is both $out[B_i]$ and $in[B_{i+1}]$. Technically, B_1 is not supposed to have two statements, because we have assumed blocks are single statements in this section. However, we are not concerned with what happens before the end of B_1 , because all variables can have any type there.

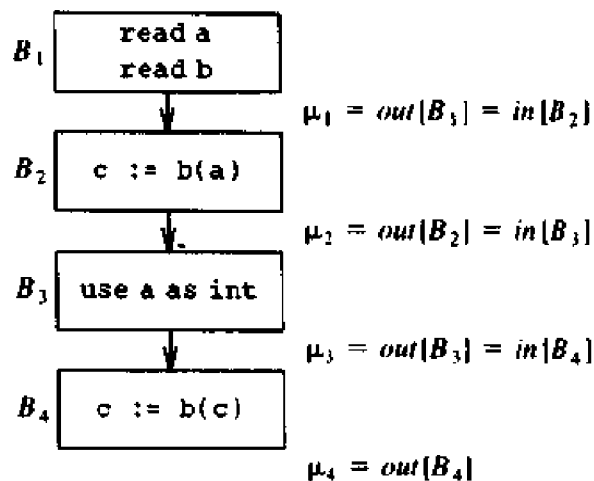


Fig. 10.66. Example program.

It turns out that we need five passes before convergence occurs and another two to detect that convergence has occurred. These passes are summarized in Fig. 10.67(a)–(e). The first pass is forward. When considering B_2 , we discover that b cannot be an integer, because it is used as a mapping. We also discover that a is used as an integer in B_3 , and therefore can only be mapped to int in μ_3 and μ_4 . These observations are summarized in Fig. 10.67(a).

	a	b	c
μ_1	11111	11111	<i>undef</i>
μ_2	11111	01111	11111
μ_3	10000	01111	11111
μ_4	10000	01111	11111

(a) FORWARD

	a	b	c
μ_1	10000	01100	<i>undef</i>
μ_2	10000	01111	11111
μ_3	10000	01111	11111
μ_4	10000	01111	11111

(b) BACKWARD

	a	b	c
μ_1	10000	01100	<i>undef</i>
μ_2	10000	01100	11111
μ_3	10000	01100	11111
μ_4	10000	01100	11111

(c) FORWARD

	a	b	c
μ_1	10000	01000	<i>undef</i>
μ_2	10000	01100	10000
μ_3	10000	01100	10000
μ_4	10000	01100	11111

(d) BACKWARD

	a	b	c
μ_1	10000	01000	<i>undef</i>
μ_2	10000	01000	10000
μ_3	10000	01000	10000
μ_4	10000	01000	10000

(e) FORWARD

Fig. 10.67. Simulation of Algorithm 10.19 on flow graph of Fig. 10.66.

The second pass, shown in Fig. 10.67(b), is backward. On this pass, when considering B_2 , we know that a must be an integer when b is applied to it. Thus, the type of b could only be $int \rightarrow int$ or $int \rightarrow func$. On the third pass, which is forward, this restriction on the type of b propagates all the way down the flow graph, as shown in Fig. 10.67(c).

The fourth pass, backward, is shown in Fig. 10.67(d). Here, the fact that c is an argument of b in B_3 tells us that c can only be an integer. Also, when we consider B_2 , we find that the result of $b(a)$ can only be of the type of c , which is int . This fact rules out the possibility that b is of type $int \rightarrow func$. Finally, in Fig. 10.67(e), we see how on the fifth pass, forward, these facts about b and c propagate. On further passes, no new inferences can be made. In this case, we have reduced the sets of possible types to single types for each variable at each point; a and c are integers, and b is a mapping from integers to integers. In general, we might be left with several possible types for a variable at a point. $\square$

10.13 SYMBOLIC DEBUGGING OF OPTIMIZED CODE

A *symbolic debugger* is a system that allows us to look at a program's data while that program is running. The debugger is usually called when a program error, such as an overflow, occurs or when certain statements, indicated by the programmer in the source code, are reached. Once invoked, the symbolic debugger allows the programmer to examine, and possibly change, any of the variables that are currently accessible to the running program.

In order for a user command like "show me the current value of a" to be intelligible to the debugger, it must have available certain information.

1. There must be a way to associate an identifier like *a* with the location it represents. Thus, the portion of the symbol table that assigns to each variable a location, e.g., a place in a global data area or in an activation record for some procedure, must be recorded by the compiler and preserved for the debugger to use. For example, this information might be encoded into the load module for the program.
2. There must be scope information, so we can disambiguate references to an identifier that is declared more than once, and so that we can tell, given we are in some procedure *p*, what other procedures' data is accessible and how do we find that data on the stack or other run-time structure. Again, this information must be taken from the symbol table of the compiler and preserved for future use by the debugger.
3. We must know where we are in the program when the debugger is invoked. This information is embedded by the compiler in the call to the debugger when the compiler handles a user-declared invocation of the debugger. It is also obtained from the exception handler when a run-time error causes the debugger to be called.
4. In order that program-location information, mentioned in (3), make sense to the user, there must be a table associating each machine language statement with the source statement from which it came. This table can be prepared by the compiler as it generates code.

While the design of a symbolic debugger is interesting in its own right, we shall consider only the difficulties that occur when trying to write a symbolic debugger for an optimizing compiler. At first glance, it might seem that there is no need to debug an optimized program. In the normal development cycle, as the user debugs a program, a fast, nonoptimizing compiler is used until the user is satisfied that the source program is correct. Only then is an optimizing compiler used.

Unfortunately, a program may run correctly with a nonoptimizing compiler, and then fail, with the same input data, when compiled by the optimizing compiler. For example, a bug may exist in the optimizing compiler, or, by reordering operations, the optimizing compiler may introduce an overflow or underflow. Also, even "nonoptimizing" compilers may do some simple

transformations, like elimination of local common subexpressions or reordering of code within a basic block, that make a big difference in how hard it is to design a symbolic debugger. Thus, we need to consider what algorithms and data structures to use in a symbolic debugger for an optimizing compiler that transforms basic blocks in arbitrary ways.

Deducing Values of Variables in Basic Blocks

For simplicity, assume that both the source and object code are sequences of intermediate statements. Treating the source as intermediate code presents no problems, since the latter is more general than the former. For example, the user may only be allowed to put breaks (calls to the debugger) between source statements, but here we allow breaks after any intermediate statement. Treating the object code as intermediate code is questionable only if the optimizer breaks a single intermediate statement into several machine statements that get separated. For example, for some reason, we may compile the two intermediate statements

```
u := v + w
x := y + z
```

into code where the two additions are performed in different registers and interleaved. If that is the case, we can treat the loads and stores of registers as if the registers were temporaries in intermediate code, for example:

```
r1 := v
r2 := y
r1 := r1 + w
r2 := r2 + z
u := r1
x := r2
```

Several problems occur when interacting with the user about a block, where the user thinks the source block is being executed, but in fact, an optimized version of that block is running:

1. Suppose we are executing the program that results from "optimizing" some basic block of the source program, and while executing statement $a := b + c$, an overflow occurs. We must tell the user that an error has occurred in one of the source statements. Since $b + c$ may be a common subexpression appearing in two or more of the source statements, to which statement do we charge the error?
2. A harder problem occurs if the user of the debugger wants to see the "current" value of some variable d . In the optimized program, d may last have been assigned at some statement s . But in the source program, s may come after the statement at which the debugger was invoked, so the value of d that is available to the debugger is not the one that the user thinks is the "current" value of d according to the listing of the

source code. Similarly, s may precede the statement invoking the debugger, but in the source there is another assignment to d between them, so the value of d available to the debugger is out of date. Is it possible to make the correct value of d available to the user? For example, could it be the value of some other variable in the optimized version, or could it be computed from the values of other variables?

3. Finally, if the user places a break after some statement of the source code, when should control be given to the debugger during the running of the optimized code?

One solution might be to run the unoptimized version of the block along with the optimized version, to make the correct value of each variable available at all times. We reject this "solution," because the subtlest of bugs, especially compiler-introduced bugs, may disappear when the instructions that caused the problem are separated from one another in time or space.

The solution we adopt is to provide enough information about each block to the debugger that it can at least answer the question: is it possible to provide the correct value of variable a , and if so, how? The structure we use to embody this information is the dag of the basic block, annotated with information about which variables hold the value corresponding to a node in the dag, at what times in both the source and optimized programs. The notation

$a: i-j$

attached to a node means that the value represented by that node is stored in variable a from the beginning of statement i through the portion of statement j just before its assignment occurs. If $j = \infty$, then a holds this value until the end of the block.

Example 10.53. In Fig. 10.68(a) we see a basic block of source code, and in Fig. 10.68(b) is a possible "optimized" version of that code. Figure 10.69 shows the dag for either block, with indications of the ranges in which the variables hold those values in both the source and optimized code. Primes are used to indicate that a statement range is in the optimized code. For example, the node labeled $+$ is the value of c in the source code from the beginning of statement (2) until just before the assignment in statement (3). It is also the value of d in the source from the beginning of statement (3) until the end. In addition, the same node is the value of d in the optimized code from statement (2') until the end. $\square$

Now we can answer the first question raised above. Suppose that an error, such as overflow, occurs while executing statement j' of the optimized code. Since the same value would be computed by any source statement that computes the same dag node as statement j' , it makes sense to report to the user that the error occurred at the first source statement computing this node. Thus, in Example 10.53, if the error occurred at statement 1', 2', 3', or 4', we would report that it occurred at statement 1, 5, 3, and 6, respectively. No

(1) $c := a+b$	(1') $d := a+b$
(2) $d := c$	(2') $t := b*e$
(3) $c := c-e$	(3') $a := d-e$
(4) $a := d-e$	(4') $b := d/t$
(5) $b := b*e$	(5') $c := a$
(6) $b := d/b$	
(a)	(b)

Fig. 10.68. Source and optimized code.

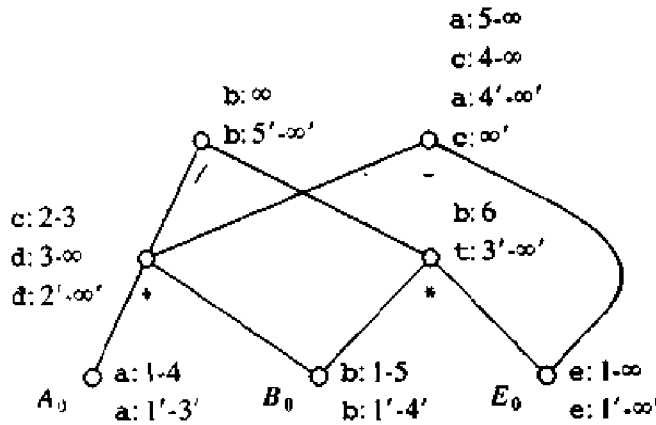


Fig. 10.69. Annotated dag.

error can occur at statement 5', because no value is computed. We defer details of how corresponding statements are computed until Example 10.54, below.

We can also answer the second question. Suppose we are at statement j' of the optimized code, and the user is told that control is at statement i of the source, where an error occurred. If the user asks to see the value of a variable x , we must find a variable y (frequently, but not always, y is x) such that the value of x at statement i of the source is the same dag node as y at statement j' in the optimized code. We inspect the dag to see what node represents the value of x at i , and we may read off from that node all the variables of the object program that ever have that value, to see if one of them holds this value at j' .

If so, we are done; if not, we may still compute the value of x at i from other variables at j' . Let n be the node for x at time i . Then we may consider the children of n , say m and p , to see if both these nodes represent the value of some variable at time j . If, say, there is a variable for m , but none for p , we may consider the children of p , recursively. Ultimately, we either find a way to compute the value of x at time i or conclude that there is no such way. If we find a way to compute the values of m and p , then we

compute them and apply the operator at n to compute the value of x at time i .¹⁷

Example 10.54. Suppose that when executing the code of Fig. 10.68(b), an error occurs at statement 2'. The statement was computing the node labeled $*$ in Fig. 10.69, and the first source statement computing that value is statement 5. Thus we report an error in statement 5.

In Fig. 10.70, we have tabulated the dag node corresponding to each variable in both the source and optimized code, at the beginning of statements 5 and 2', respectively; nodes are indicated by their label, either an operator symbol or an initial value symbol like A_0 . We also indicate how to compute the value at time 5 from the values of variables at time 2'. For example, if the user asks for the value of a , the value of the node labeled $-$ is given. No variable has that value at time 2', but fortunately, there are variables, d and e , that hold the value of each of the children of node $-$ at time 2', so we may print the value of a by computing the value of $d-e$. $\square$

VARIABLE	VALUE AT		OBTAIN BY
	TIME 2'	TIME 5	
a	A_0	$-$	$d-e$
b	B_0	B_0	b
c	undefined	$-$	$d-e$
d	$+$	$+$	d
e	E_0	E_0	e
t	undefined		

Fig. 10.70. Values of variables at times 2' and 5.

Now let us answer the third question: how to handle debugger calls inserted by the user. In a sense the answer is trivial; if the user asks for a break after statement i in the source program, we can halt execution at the beginning of the block. If the user wants to see the value of some variable x after statement i , we consult the annotated dag to see what node represents the desired value of x , and we compute that value from the initial values of variables for that block.

On the other hand, we can leave less work for the debugger, and also avoid

¹⁷ A subtlety occurs if the computation of the value of node n causes another error. We must then report to the user that the error actually occurred earlier, at the first source statement computing the value of n .

some situations where attempts to compute a value lead to errors that must be announced to the user, if we postpone the call to the debugger to as late a time as possible. It is easy to compute the last statement j' in the optimized program, such that we call the debugger after j' and pretend to the user that the call was made after statement i of the source program. To find j' , let S be the set of nodes of the dag that correspond to the value of some variable of the source program immediately after statement i . We may be asked by the user to compute any value in S . Thus, we may break after statement j' of the optimized code only if for each node n in S there is some $k' > j'$ such that some variable is associated with node n at time k' in the optimized code. For then we know that the value of n is either available immediately after statement j' , or it will be computed sometime after statement j' . In the former case, it is trivial to compute the value of n if we break after j' , while in the latter case, we know that the values available after j' are sufficient to compute n somehow.

Example 10.55. Consider the source and optimized code of Fig. 10.68 again, and suppose that the user inserts a break after statement (3) of the source code. To find the set S , inspect the dag of Fig. 10.69, and see which nodes have source-program variables attached to them at time 4. These nodes are the ones labeled A_0 , B_0 , E_0 , $+$, and $-$ in that figure.

Now, we look at the dag again to find the largest j' such that each of the nodes in S has some variable of the optimized code attached to it at a time strictly greater than j' . The nodes labeled $+$, $-$, and E_0 present no problem, since their values are carried by variables d , a , and e , respectively, at time ∞' . Nodes A_0 and B_0 do limit the value of j' , and the earliest of these to lose its value is A_0 , whose value is destroyed by statement $3'$. Thus, $j' = 2'$ is the largest possible value of j' ; that is, if the user asks for a break after source statement 3, we can give him the break after statement $2'$. $\square$

The reader should be aware of a subtlety in Example 10.55, for which no really good solution exists. If we run the optimized code through statement $2'$ before we call the debugger, an error in the computation of $b * e$ at statement $2'$ (for example, an underflow) may cause the debugger to be invoked before the intended call. However, since the computation corresponding to statement $2'$ doesn't occur until statement 5 in the source program, we are going to tell the user that the error occurred at statement 5. It will be somewhat mysterious to the user how we got to statement 5 without calling the debugger at statement 3. Probably the best solution to this problem is not to allow j' to be so large that there is a statement k' of the optimized code, with $k' \leq j'$, such that the source code does not compute the value computed by k' until after the statement i at which the break was placed.

Effects of Global Optimization

When our compiler performs global optimizations, there are harder problems

for the symbolic debugger to solve, and frequently, no way to find the correct value of a variable at a point exists. Two important transformations that do not cause significant trouble are induction-variable elimination and global common subexpression elimination; in each case the problem can be localized to a few blocks and treated in the manner discussed above.

Induction-Variable Elimination

If we eliminate a source program induction variable i in favor of some member of i 's family, say t , then there is some linear function relating i and t . Moreover, if we follow the methods of Section 10.7, the optimized code will change t in exactly those blocks in which i is changed, so the linear relationship between i and t always holds. Thus, after taking into account reordering of statements within a block that assigns to t (and in the source, assigns to i), we can provide the user with the "current" value of i by a linear transformation on t .

We have to be careful if i is not defined prior to the loop, since t surely will be assigned before loop entry, and we might thus provide a value for i at a point in the program where the user expects i to be undefined. Fortunately, it is usual for a source-program variable that is an induction variable to be initialized prior to the loop, and only compiler-generated variables (whose values the user cannot ask for) will be undefined on entry to the loop. If that is not the case for some induction variable i , then we have a problem similar to that for code motion, to be discussed below.

Global Common Subexpression Elimination

If we perform a global common subexpression elimination for expression $a+b$, we also affect a limited number of blocks in simple ways. If t is the variable used to hold the value of $a+b$, then at certain blocks computing $a+b$ we might replace

```
c := a+b
```

by

```
t := a+b
```

```
c := t
```

This sort of change can be handled by the methods for basic blocks already discussed.

In other blocks, a use such as $d:=a+b$ may be replaced by a use $d:=t$. To handle this situation by previous methods, we have only to note in the dag for this block that the value of t remains for all time at the value of the node for $a+b$ (which will appear in the dag for the source code, but would not otherwise appear in the dag for the optimized code).

Code Motion

Other transformations are not so easy to handle. For example, suppose we move a statement

$s: a := b + c$

out of a loop because it is loop-invariant. If we call the debugger within the loop, we do not know whether statement s would have been executed yet in the source program, and thus we cannot know whether the current value of a is the one the user would see in the source program.

One possibility is to insert into the optimized code a new variable that, within the loop, indicates whether a has been assigned in the loop, (which can only occur at the old position of statement s). However, this strategy may not always be suitable, because for absolute reliability, only the real code, not a version of the code constructed specially for debugging purposes, should be used.

There is a common special case where we can do better, however. Suppose that the block B containing statement s in the source program divides the loop into two sets of nodes: those that dominate B and those dominated by it. Moreover, assume that all predecessors of the header are dominated by B , as suggested in Fig. 10.71. Then the first time through the blocks that dominate B , we may assume a has not been assigned in the loop, while the first time through the blocks that B dominates, a has been assigned at statement s . Of course, the second and subsequent times through the loop, a has surely been assigned at s .

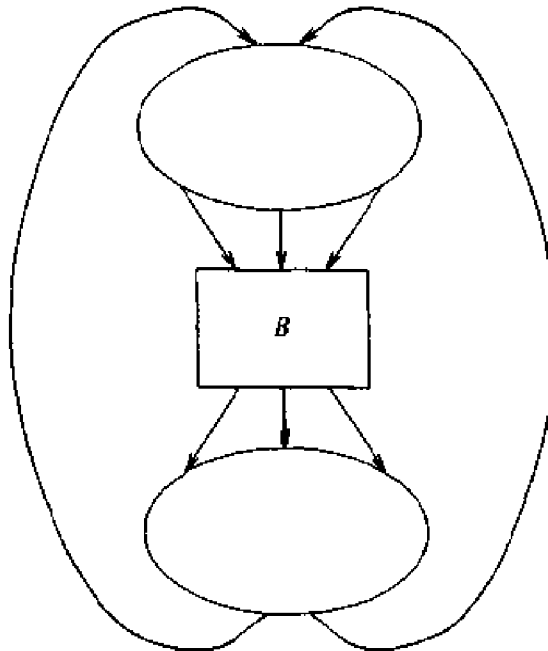


Fig. 10.71. A block that divides a loop into two parts.

If the call to the debugger is caused by a run-time error, there is a good chance that the error is exposed the first time through the loop. If that is the case, then we have only to know whether we are above B or below it in Fig. 10.71. Then we know whether the value of a is that defined at s , in which case we can just print the value of a produced by the optimized code, or whether the value of a is that which a had on entry to the loop in the source version of the program. In the latter case, there is little we can do, except if

1. the debugger has reaching definition information available to it, for both the source and optimized program,
2. there is a unique definition of a that reaches the header in the source program, and
3. that definition is also the unique definition of some variable x that reaches the point where the debugger was called.

If all these conditions hold, then we can print the value of x and say it is the value of a .

The reader should be aware that this line of reasoning does not hold if the debugger was called by a user-inserted break point, for then we cannot have any reason to suspect that we are going around the loop for the first time. However, if user-inserted breaks are being used, then it might also be reasonable to insert code into the optimized program to help the debugger tell whether this is the first time through the loop, a solution that we mentioned earlier but suggested might not be suitable because it tampered with the optimized code.

EXERCISES

10.1 Consider the matrix multiplication program in Fig. 10.72.

```

begin
  for i := 1 to n do
    for j := 1 to n do
      c[i,j] := 0;
    for i := 1 to n do
      for j := 1 to n do
        for k := 1 to n do
          c[i,j] := c[i,j] + a[i,k] * b[k,j]
        end
      end
    end
  end
end

```

Fig. 10.72. Matrix multiplication program.

- a) Assuming a , b , and c are allocated static storage and there are four bytes per word in a byte-addressed memory, produce three-address statements for the program in Fig. 10.72.

- b) Generate target-machine code from the three-address statements.
 - c) Construct a flow graph from the three-address statements.
 - d) Eliminate the common subexpressions from each basic block.
 - e) Find the loops in the flow graph.
 - f) Move the loop-invariant computations out of the loops.
 - g) Find the induction variables of each loop and eliminate them where possible.
 - h) Generate target-machine code from the flow graph in (g). Compare the code with that produced in (b).
- 10.2** Compute reaching definitions and ud-chains for the original flow graph from Exercise 10.1(c) and the final flow graph from 10.1(g).
- 10.3** The program of Fig. 10.73 counts the primes from 2 to n using the sieve method on a suitably large array.

```

begin
    read n;
    for i := 2 to n do
        a[i] := true;    /* initialize */
    count := 0;
    for i := 2 to n ** .5 do
        if a[i] then    /* i is a prime */
            begin
                count := count + 1;
                for j := 2 * i to n by i do
                    a[j] := false
                    /* j is divisible by i */
                end;
            print count
        end
    end
end

```

Fig. 10.73. Program to calculate primes.

- a) Translate the program of Fig. 10.73 into three-address statements assuming a is allocated static storage.
- b) Generate target-machine code from the three-address statements.
- c) Construct a flow graph from the three-address statements.
- d) Show the dominator tree for the flow graph in (a).
- e) For the flow graph in (c), indicate the back edges and their natural loops.
- f) Move the invariant computations out of loops using Algorithm 10.7.
- g) Eliminate induction variables wherever possible.

- h) Propagate out copy statements wherever possible.
- i) Is loop jamming possible? If so, do it.
- j) On the assumption that n will always be even, unroll inner loops once each. What new optimizations are now possible?

10.4 Repeat Exercise 10.3 on the assumption that a is allocated dynamic storage, with ptr a pointer to the first word of a .

10.5 For the flow graph of Fig. 10.74 compute:

- a) ud- and du-chains,
- b) live variables at the end of each block,
- c) available expressions.

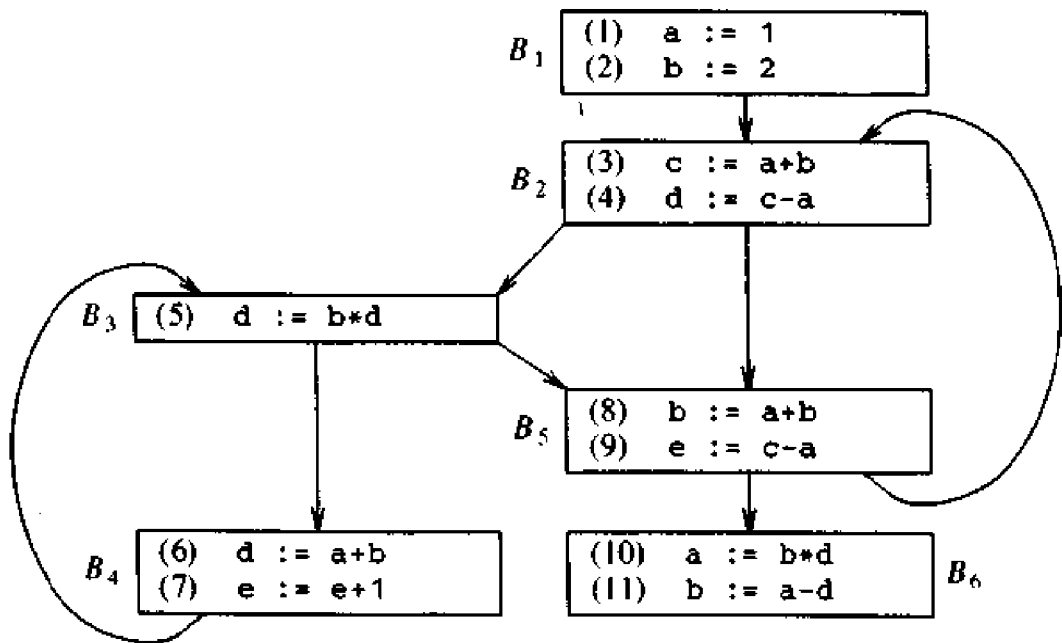


Fig. 10.74. Flow graph.

- 10.6** Is any constant folding possible in Fig. 10.74? If so, do it.
- 10.7** Are there any common subexpressions in Fig. 10.74? If so, eliminate them.
- 10.8** An expression e is said to be *very busy* at point p if no matter what path is taken from p , the expression e will be evaluated before any of its operands are defined. Give a data-flow algorithm in the style of Section 10.6 to find all very busy expressions. What confluence operator do you use, and does propagation run forward or backward? Apply your algorithm to the flow graph in Fig. 10.74.

- *10.9 If expression e is very busy at point p we can *hoist* e by computing it at p and preserving its value for subsequent use. (Note: This optimization does not usually save time, but may save space.) Give an algorithm to hoist very busy expressions.
- 10.10 Are there any expressions that may be hoisted in Fig. 10.74? If so, hoist them.
- 10.11 Where possible, propagate out any copy steps introduced in the modifications of Exercises 10.6, 10.7, and 10.10.
- 10.12 An *extended basic block* is a sequence of blocks $B_1, \dots, B_k$ such that for $1 \leq i < k$, B_i is the only predecessor of B_{i+1} , and B_1 does not have a unique predecessor. Find the extended basic block ending at each node in
 - a) Fig. 10.39,
 - b) the flow graph constructed in Exercise 10.1(c),
 - c) Fig. 10.74.
- *10.13 Give an algorithm that runs in time $O(n)$ on an n -node flow graph to find the extended basic block ending at each node.
- 10.14 We can do some interblock code optimization without any data-flow analysis by treating each extended basic block as if it were a basic block. Give algorithms to do the following optimizations within an extended basic block. In each case, indicate what effect on other extended basic blocks a change within one extended basic block can have.
 - a) Common subexpression elimination.
 - b) Constant folding.
 - c) Copy propagation.
- 10.15 For the flowgraph of Exercise 10.1(c):
 - a) Find a sequence of T_1 and T_2 reductions for the graph.
 - b) Find the sequence of interval graphs.
 - c) What is the limit flow graph? Is the flow graph reducible?
- 10.16 Repeat Exercise 10.15 for the flow graph of Fig. 10.74.
- **10.17 Show that the following conditions are equivalent (they are alternative definitions of "reducible flow graph").
 - a) The limit of T_1 - T_2 reduction is a single node.
 - b) The limit of interval analysis is a single node.
 - c) The flow graph can have its edges divided into two classes; one forms an acyclic graph and the other, the "back" edges, consists of edges whose heads dominate their tails.
 - d) The flow graph has no subgraph of the form shown in Fig. 10.75. Here, n_0 is the initial node, and n_0, a, b , and c are all distinct, with the exception that $a = n_0$ is possible. The arrows represent

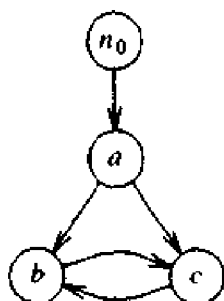


Fig. 10.75. Forbidden subgraph for reducible flow graphs.

node-disjoint paths (except for the endpoints, of course).

- 10.18** Give algorithms to compute (a) available expressions and (b) live variables for the language with pointers discussed in Section 10.8. Be sure to make conservative assumptions about *gen*, *kill*, *use*, and *def* in (b).
- 10.19** Give an algorithm to compute reaching definitions interprocedurally using the model of Section 10.8. Again, be sure you make conservative approximations to the truth.
- 10.20** Suppose parameters are passed by value instead of by reference. Can two names be aliases of one another? What if copy-restore linkage is used?
- 10.21** What is the depth of the flow graph in Exercise 10.1(c)?
- **10.22** Prove that the depth of a reducible flow graph is never less than the number of times interval analysis must be performed to produce a single node.
- *10.23** Generalize the structure-based data-flow analysis algorithm of Section 10.8 to a general data-flow framework in the sense of Section 10.11. What assumptions about F and $\wedge$ do you have to make to ensure that your algorithm works?
- *10.24** An interesting and powerful data-flow analysis framework is obtained by imagining that the “values” to be propagated are all the possible partitions of expressions so that two expressions are in the same class only if they have the same value. To avoid having to list all of the infinity of possible expressions, we can represent such values by listing only the minimal expressions that are equivalent to some other expression. For example, if we execute the statements

```

A := B
C := A + D

```

then we have the following minimal equivalences: $A \equiv B$, and

$C \equiv A + D$. From these follow other equivalences, such as $C \equiv B + D$ and $A + E \equiv B + E$, but there is no need to list these explicitly.

- a) What is the appropriate meet, or confluence, operator for this framework?
- b) Give a data structure to represent values and an algorithm to implement the meet operator.
- c) What are the appropriate functions to associate with statements? Explain the effect that the function associated with assignments such as $A := B + C$ should have on a partition.
- d) Is this framework distributive? Monotone?

10.25 How would you use the data gathered by the framework of Exercise 10.24 to perform

- a) common subexpression elimination?
- b) copy propagation?
- c) constant folding?

***10.26** Give formal proofs of the following about the relation $\leq$ on lattices.

- a) $a \leq b$ and $a \leq c$ implies $a \leq (b \wedge c)$.
- b) $a \leq (b \wedge c)$ implies $a \leq b$.
- c) $a \leq b$ and $b \leq c$ implies $a \leq c$.
- d) $a \leq b$ and $b \leq a$ implies $a = b$.

****10.27** Show that the following condition is necessary and sufficient for the iterative data-flow algorithm, with depth-first ordering, to converge within 2 plus the depth passes. For all functions f and g and value a :

$$f(g(a)) \geq f(a) \wedge g(a) \wedge a$$

10.28 Show that the reaching definitions and available expressions frameworks satisfy the condition of Exercise 10.27. Note: In fact, these frameworks converge in 1 plus the depth passes.

****10.29** Is the condition of Exercise 10.27 implied by monotonicity? By distributivity? Vice versa?

10.30 We see in Fig. 10.76 two basic blocks, the first "initial" code and the second an optimized version.

- a) Construct the dags for the blocks of Fig. 10.76(a) and (b). Verify that, on the assumption only J is live on exit, these two blocks are equivalent.
- b) Annotate the dag with the times at which each variable's value is known at each node.
- c) Indicate, for an error occurring at each of statements (1') through (4'), at which statement of Fig. 10.76(a) we should say an error occurs.

1) $E := A+B$	1') $E := A+B$
2) $F := E-C$	2') $E := E-C$
3) $G := F*D$	3') $F := E*D$
4) $H := A+B$	4') $J := E+F$
5) $I := I-C$	
6) $J := I+G$	
(a) INITIAL	(b) OPTIMIZED

Fig. 10.76. Initial and optimized code.

- d) For each of the errors in part (c), indicate for which variables of Fig. 10.76(a) it is possible to compute a value, and how we do so.
- e) Suppose we allowed ourselves to use algebraically valid laws like "if $a + b = c$ then $a = c - b$." Would your answer to (d) change?
- 10.31** Generalize Example 10.14 to account for an arbitrary set of break statements. Also, generalize it to allow continue statements, which do not break the inner loop but proceed directly to the next iteration of the loop. Hint: Use the techniques developed in Section 10.10 for reducible flow graphs.
- 10.32** Show that in Algorithm 10.3 the *in* and *out* sets of definitions never decrease. Similarly, show that in Algorithm 10.4 these sets of expressions never increase.
- 10.33** Generalize Algorithm 10.9 for the elimination of induction variables to the case where multiplicative constants can be negative.
- 10.34** Generalize the algorithm for determining what pointers can point to, from Section 10.8, to the case where pointers are permitted to point to other pointers.
- *10.35** When estimating each of the following sets, tell whether too-large or too-small estimates are conservative. Explain your answer in terms of the intended use of the information.
- Available expressions.
 - Variables changed by a procedure.
 - Variables not changed by a procedure.
 - Induction variables belonging to a given family.
 - Copy statements reaching a given point.
- *10.36** Refine Algorithm 10.12 to compute the aliases of a given variable at a given point.
- *10.37** Modify Algorithm 10.12 for the cases that parameters are passed

- a) by value,
 - b) by copy-restore.
- *10.38** Prove that Algorithm 10.13 converges to a superset (not necessarily proper) of the truly changed variables.
- *10.39** Generalize Algorithm 10.13 to determine changed variables in the case that procedure-valued variables are permitted.
- *10.40** Prove that in each interval graph each node represents a region of the original flow graph.
- 10.41** Prove that Algorithm 10.16 correctly computes the set of dominators of each node.
- *10.42** Modify Algorithm 10.17 (structure-based reaching definitions) to compute reaching definitions for designated small regions only, without requiring that the whole flow graph be present in memory at once. Be sure your result is conservative. Adapt your algorithm to available expressions. Which is more likely to provide useful information?
- *10.43** In Section 10.10 we proposed a speedup to Algorithm 10.17 based on combining a T_1 with a T_2 reduction. Prove the correctness of the modification.
- 10.44** Generalize the iterative method of Section 10.11 to backward-flowing problems.
- **10.45** Prove that when Algorithm 10.18 converges, the resulting solution is $\leq$ the *mop* solution by showing that for every path P of length i , then after i iterations, $in[B_i] \leq f_P[\top]$.
- 10.46** In Fig. 10.77 is a flow graph of a program in the hypothetical language introduced in Section 10.12. Find the best estimate of the types of each variable using Algorithm 10.19.

BIBLIOGRAPHIC NOTES

Additional information about code optimization may be found in Cocke and Schwartz [1970], Abel and Bell [1972], Schaefer [1973], Hecht [1977], and Muchnick and Jones [1981]. Allen [1975] provides a bibliography on program optimization.

Many optimizing compilers are described in the literature. Ershov [1966] discusses an early compiler that used sophisticated optimization techniques. Lowry and Medlock [1969] and Scarborough and Kolsky [1980] detail the construction of optimizing compilers for Fortran. Busam and Englund [1969] and Metcalf [1982] describe additional techniques for Fortran optimization. Wulf et al. [1975] discuss the design of an influential optimizing compiler for Bliss.

Allen et al. [1980] describe a system that was built to experiment with

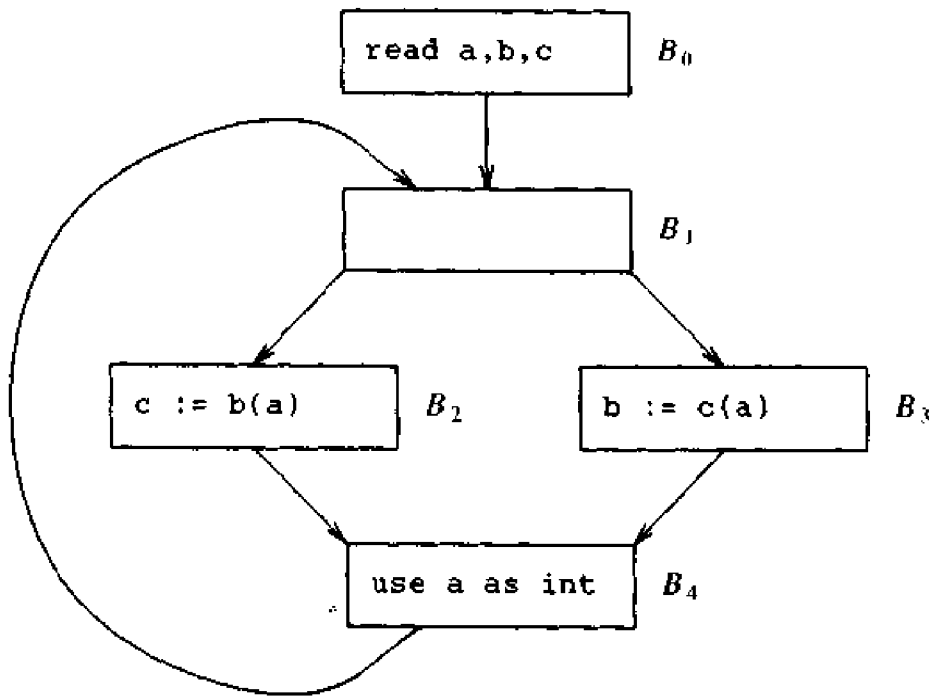


Fig. 10.77. Example program for type inference.

program optimizations. Cocke and Markstein [1980] report on the effectiveness of various optimizations for a PL/I-like language. Anklam, Cutler, Heinen and MacLaren [1982] describe the implementation of optimizing transformations that were used in compilers for PL/I and C. Auslander and Hopkins [1982] report on a compiler for a variant of PL/I that uses a simple algorithm to produce low-level intermediate code that is then improved by global optimizing transformations. Freudenberger, Schwartz, and Sharir [1983] describe experiences with an optimizer for SETL. Chow [1983] reports on experiments with a portable, machine-independent, optimizer for Pascal. Powell [1984] describes a portable, machine-independent, optimizing compiler for Modula-2.

The systematic study of data-flow analysis techniques begins with Allen [1970] and Cocke [1970], since published together as Allen and Cocke [1976], although various data-flow analysis methods were in use before then.

Syntax-directed data-flow analysis, as introduced in Section 10.5, has been used in Bliss (Wulf et al. [1975], Geschke [1972]), SIMPL (Zelkowitz and Bail [1974]), and Modula-2 (Powell [1984]). Additional discussions of this family of algorithms appears in Hecht and Schaffer [1975], Hecht [1977], and Rosen [1977].

The iterative approach to data-flow analysis discussed in Section 10.6 has been traced to Vyssotsky (see Vyssotsky and Wegner [1963]), who used the method in a 1962 Fortran compiler. The use of depth-first ordering to

improve the efficiency is from Hecht and Ullman [1975].

The interval-analysis approach to data-flow analysis was pioneered by Cocke [1970]. Kennedy [1971] originates the use of interval analysis for backward flow problems, like live variables. There is reason to believe, following Kennedy [1976], that interval-based methods are somewhat more efficient than iterative ones, if the language being optimized tends to produce few, if any, nonreducible flow graphs. The variant used here, based on T_1 and T_2 , is from Ullman [1973]. A somewhat faster version that takes advantage of the fact that most regions have a single exit was given in Graham and Wegman [1976].

The original definition of a reducible flow graph, one that becomes a single node under iterated interval analysis, is from Allen [1970]. Equivalent characterizations are found in Hecht and Ullman [1972, 1974], Kasyanov [1973], and Tarjan [1974b]. Node splitting for nonreducible flow graphs is from Cocke and Miller [1969].

The idea that structured flow of control is modeled by reducible flow graphs is expressed in Kosaraju [1974], Kasami, Peterson, and Tokura [1973], and Cherniavsky, Henderson, and Keohane [1976]. Baker [1977] describes their use in a program-structuring algorithm.

The lattice-theoretic approach to iterative data-flow analysis began with Kildall [1973]. Tennenbaum [1974] and Wegbreit [1975] are similar formulations. The efficient version of Kildall's algorithm, in which depth-first order is used, is from Kam and Ullman [1976].

While Kildall assumed the distributivity condition (which his frameworks such as the constant computation framework of Example 10.42 do not actually satisfy), the adequacy of monotonicity was perceived in a number of papers giving data-flow algorithms, such as Tennenbaum [1974], Schwartz [1975a, b], Graham and Wegman [1976], Jones and Muchnick [1976], Kam and Ullman [1977], and Cousot and Cousot [1977].

Since different algorithms require differing assumptions about the data, the theory of what properties are needed for certain algorithms was developed by Kam and Ullman [1977], Rosen [1980], and Tarjan [1981].

Another direction that followed from Kildall's paper was improving the algorithms for dealing with the particular data-flow problems (e.g., Example 10.42) he introduced. One key idea is that the lattice elements need not be treated as atomic, but we can exploit the fact that they are really mappings from variables to values. See Reif and Lewis [1977] and Wegman and Zadeck [1985]. Also, Kou [1977] exploits the idea for more conventional problems.

Kennedy [1981] is a survey of data-flow analysis techniques, and Cousot [1981] surveys lattice-theoretic ideas.

Gear [1965] introduced the basic loop optimizations of code motion and a limited form of induction-variable elimination. Allen [1969] is a fundamental paper on loop optimization; Allen and Cocke [1972] and Waite [1976b] are more extensive surveys of techniques in the area. Morel and Renvoise [1979] describe an algorithm that simultaneously eliminates redundant and invariant

computations from loops.

The discussion of induction-variable elimination in Section 10.7 is based on Lowry and Medlock [1969]. See Allen, Cocke, and Kennedy [1981] for more powerful algorithms.

An algorithm for some of the loop problems not discussed in detail here, such as finding whether there is a path from a to b that does not go through c , can be solved by an efficient algorithm of Wegman [1983].

The use of dominators, both for loop discovery and for performing code motion was pioneered by Lowry and Medlock [1969], although they attribute the general idea to Prosser [1959]. Algorithm 10.16 for finding dominators was discovered independently by Purdom and Moore [1972] and Aho and Ullman [1973a]. The use of depth-first ordering to speed the algorithm is from Hecht and Ullman [1975], while the asymptotically most efficient way to do the job is from Tarjan [1974a]. Lengauer and Tarjan [1979] describe an efficient algorithm for finding dominators that is suitable for practical use.

The study of aliasing and interprocedural analysis begins with Spillman [1971] and Allen [1974]. Some more powerful methods than those of Section 10.8 have been developed. In general, they deal with the alias relation at each point of the program to avoid some of the impossible alias pairs that our simple algorithm "discovers." These works include Barth [1978], Banning [1979], and Weihl [1980]. See also Ryder [1979] on the construction of calling graphs.

An issue similar to interprocedural analysis, the effect of exceptions on the data-flow analysis of programs, is discussed by Hennessy [1981].

The fundamental paper of determination of types by data-flow analysis is Tennenbaum [1974], on which our discussion of Section 10.12 is based. Kaplan and Ullman [1980] give a more powerful algorithm for type detection.

The discussion of symbolic debugging of optimized code in Section 10.13 is from Hennessy [1982].

There have been a number of papers that attempt to evaluate the improvement due to various optimizations. The value of an optimization appears to be highly dependent on the language being compiled. The reader may wish to consult the classic study of Fortran optimization in Knuth [1971b], or the papers by Gajewska [1975], Palm [1975], Cocke and Kennedy [1976], Cocke and Markstein [1980], Chow [1983], Chow and Hennessy [1984], and Powell [1984].

Another topic in optimization that we have not covered here is the optimization of "very high-level" languages, such as the set-theoretic language SETL, where we are really changing the underlying algorithms and data structures. One central optimization in this area is generalized induction-variable elimination, as in Earley [1975b], Fong and Ullman [1976], Paige and Schwartz [1977], and Fong [1979].

Another key step in optimization for very high-level languages is the selection of data structures; this topic is covered in Schwartz [1975a, b], Low and Rovner [1976], and Schonberg, Schwartz, and Sharir [1981].

We also have not touched on issues in incremental code optimization, where small modifications to the program do not require a complete reoptimization. Ryder [1983] discusses incremental data-flow analysis, while Pollock and Soffa [1985] attempt to do incremental optimization of basic blocks.

Finally, we should mention some of the many other ways that data-flow analysis techniques have been used. Backhouse [1984] uses it on the transition graphs associated with parsers to perform error recovery. Harrison [1977] and Suzuki and Ishihata [1977] discuss its use in compile-time array-bounds checking.

One of the most significant uses of data-flow analysis outside code optimization is in the area of static (compile-time) checking for program errors. Fosdick and Osterweil [1976] is a fundamental paper, while Osterweil [1981], Adrion, Bronstad, and Cherniavsky [1982], and Freudenberger [1984] give some more recent developments.

Want to Write a Compiler?

Having seen the principles, techniques, and tools of compiler design, suppose we want to write a compiler. With some advance planning the implementation can proceed more quickly and smoothly than otherwise. This brief chapter raises some implementation issues that arise in compiler construction. Much of the discussion is focussed around writing compilers using the UNIX operating system and its tools.

11.1 PLANNING A COMPILER

A new compiler may be for a new source language, or produce new target code, or both. Using the framework espoused in this book, we ultimately obtain a design for a compiler that consists of a set of modules. Several diverse factors impact the design and implementation of these modules.

Source Language Issues

The “size” of a language affects the size and number of the modules. Although there is no precise definition for the size of a language, it is apparent that a compiler for a language like Ada or PL/I is bigger and harder to implement than a compiler for a little language like Ratfor (a “rational” Fortran preprocessor, Kernighan [1975]) or EQN (a language for typesetting mathematics).

Another important factor is the extent to which the source language will change during the course of compiler construction. Although the source language specification may look immutable, few languages remain the same for the life of a compiler. Even a mature language evolves, albeit slowly. Fortran, for example, has changed considerably from the language it was in 1957; the looping, Hollerith, and conditional statements in Fortran 77 are quite different from those in the original language. Rosler [1984] reports on the evolution of C.

A new, experimental language, on the other hand, may undergo dramatic change as it is implemented. One way to create a new language is to evolve a compiler for a working prototype of the language into one that satisfies the needs of a certain group of users. Many of the “little” languages initially

developed on the UNIX system like AWK and EQN were created this way.

Consequently, a compiler writer might anticipate at least some amount of change in the definition of the source language over the lifetime of a compiler. Modular design and use of tools can help the compiler writer cope with this change. For example, using generators to implement the lexical analyzer and parser of a compiler allows the compiler writer to accommodate syntactic changes in the language definition more readily than when the lexical analyzer and parser are written directly in code.

Target Language Issues

The nature and limitations of the target language and run-time environment have to be carefully considered, for they too have a strong influence on the compiler design and on what code generation strategies it should use. If the target language is new, the compiler writer is well advised to make sure that it is correct and that its timing sequences are well understood. A new machine or a new assembler may have bugs that a compiler is likely to uncover. Bugs in the target language can greatly aggravate the task of debugging the compiler itself.

A successful source language is likely to be implemented on several target machines. If a language persists, compilers for the language will need to generate code for several generations of target machines. Further evolution in machine hardware seems certain, so retargetable compilers are likely to have an edge. Consequently, the design of the intermediate language is important, as is confining machine-specific details to a small number of modules.

Performance Criteria

There are several aspects to compiler performance: compiler speed, code quality, error diagnostics, portability, and maintainability. The tradeoffs between these criteria are not so clear cut, and the compiler specification may leave many of these parameters unspecified. For example, is compilation speed of greater importance than the speed of the target code? How important are good error messages and error recovery?

Compiler speed can be achieved by reducing the number of modules and passes as much as possible, perhaps to the point of generating machine language directly in one pass. However, this approach may not produce a compiler that generates high quality target code, nor one that is particularly easy to maintain.

There are two aspects to portability: retargetability and rehostability. A retargetable compiler is one that can be modified easily to generate code for a new target language. A rehostable compiler is one that can be moved easily to run on a new machine. A portable compiler may not be as efficient as a compiler designed for a specific machine, because the single-machine compiler can safely make specific assumptions about the target machine that a portable compiler cannot.

11.2 APPROACHES TO COMPILER DEVELOPMENT

There are several general approaches that a compiler writer can adopt to implement a compiler. The simplest is to retarget or rehost an existing compiler. If there is no suitable existing compiler, the compiler writer might adopt the organization of a known compiler for a similar language and implement the corresponding components, using component-generation tools or implementing them by hand. It is relatively rare that a completely new compiler organization is required.

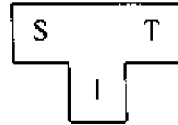
No matter what approach is adopted, compiler writing is an exercise in software engineering. Lessons from other software efforts (see for example Brooks [1975]) can be applied to enhance the reliability and maintainability of the final product. A design that readily accommodates change will allow the compiler to evolve with the language. The use of compiler-building tools can be a significant help in this regard.

Bootstrapping

A compiler is a complex enough program that we would like to write it in a friendlier language than assembly language. In the UNIX programming environment, compilers are usually written in C. Even C compilers are written in C. Using the facilities offered by a language to compile itself is the essence of *bootstrapping*. Here we shall look at the use of bootstrapping to create compilers and to move them from one machine to another by modifying the back end. The basic ideas of bootstrapping have been known since the mid 1950's (Strong et al. [1958]).

Bootstrapping may raise the question, "How was the first compiler compiled?" which sounds like, "What came first, the chicken or the egg?" but is easier to answer. For an answer we consider how Lisp became a programming language. McCarthy [1981] notes that in late 1958 Lisp was used as a notation for writing functions; they were then hand-translated into assembly language and run. The implementation of an interpreter for Lisp occurred unexpectedly. McCarthy wanted to show that Lisp was a notation for describing functions "much neater than Turing machines or the general recursive definitions used in recursive function theory," so he wrote a function *eval* $[e, a]$ in Lisp that took a Lisp expression e as an argument. S. R. Russell noticed that *eval* could serve as an interpreter for Lisp, hand-coded it, and thus created a programming language with an interpreter. As mentioned in Section 1.1, rather than generating target code, an interpreter actually performs the operations of the source program.

For bootstrapping purposes, a compiler is characterized by three languages: the source language S that it compiles, the target language T that it generates code for, and the implementation language I that it is written in. We represent the three languages using the following diagram, called a *T-diagram*, because of its shape (Bratman [1961]).



Within text, we abbreviate the above T-diagram as $S_I T$. The three languages S , I , and T may all be quite different. For example, a compiler may run on one machine and produce target code for another machine. Such a compiler is often called a *cross-compiler*.

Suppose we write a cross-compiler for a new language L in implementation language S to generate code for machine N ; that is, we create $L_S N$. If an existing compiler for S runs on machine M and generates code for M , it is characterized by $S_M M$. If $L_S N$ is run through $S_M M$, we get a compiler $L_M N$, that is, a compiler from L to N that runs on M . This process is illustrated in Fig. 11.1 by putting together the T-diagrams for these compilers.

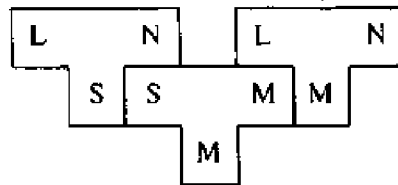
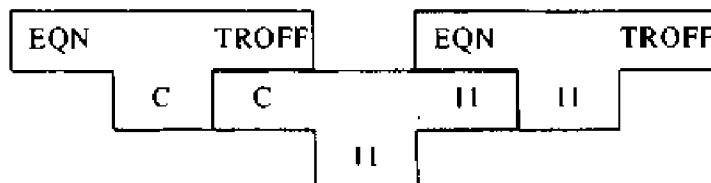


Fig. 11.1. Compiling a compiler.

When T-diagrams are put together as in Fig. 11.1, note that the implementation language S of the compiler $L_S N$ must be the same as the source language of the existing compiler $S_M M$ and that the target language M of the existing compiler must be that same as the implementation language of the translated form $L_M N$. A trio of T-diagrams such as Fig. 11.1 can be thought of as an equation

$$L_S N + S_M M = L_M N$$

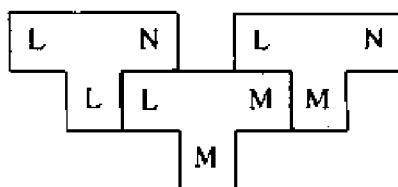
Example 11.1. The first version of the EQN compiler (see Section 12.1) had C as the implementation language and generated commands for the text formatter TROFF. As shown in the following diagram, a cross-compiler for EQN, running on a PDP-11, was obtained by running $EQN_C TROFF$ through the C compiler $C_{11 11}$ on the PDP-11.



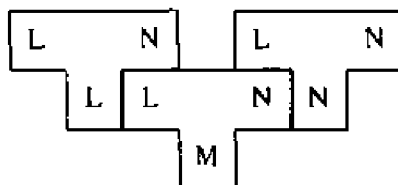
One form of bootstrapping builds up a compiler for larger and larger subsets of a language. Suppose a new language L is to be implemented on machine M . As a first step we might write a small compiler that translates a subset S of L into the target code for M ; that is, a compiler $S_M M$. We then use the subset S to write a compiler $L_S M$ for L . When $L_S M$ is run through $S_M M$, we obtain an implementation of L , namely, $L_M M$. Neliac was one of the first languages to be implemented in its own language (Huskey, Halstead, and McArthur [1960]).

Wirth [1971] notes that Pascal was first implemented by writing a compiler in Pascal itself. The compiler was then translated "by hand" into an available low-level language without any attempt at optimization. The compiler was for a subset "(>60 per cent)" of Pascal; several bootstrapping stages later a compiler for all of Pascal was obtained. Lecarme and Peyrolle-Thomas [1978] summarize methods that have been used to bootstrap Pascal compilers.

For the advantages of bootstrapping to be realized fully, a compiler has to be written in the language it compiles. Suppose we write a compiler $L_L N$ for language L in L to generate code for machine N . Development takes place on a machine M , where an existing compiler $L_M M$ for L runs and generates code for M . By first compiling $L_L N$ with $L_M M$, we obtain a cross-compiler $L_M N$ that runs on M , but produces code for N :



The compiler $L_L N$ can be compiled a second time, this time using the generated cross-compiler:



The result of the second compilation is a compiler $L_N N$ that runs on N and generates code for N . There are a number of useful applications of this two-step process, so we shall write it as in Fig. 11.2.

Example 11.2. This example is motivated by the development of the Fortran H compiler (see Section 12.4). "The compiler was itself written in Fortran and bootstrapped three times. The first time was to convert from running on the IBM 7094 to System/360 — an arduous procedure. The second time was to optimize itself, which reduced the size of the compiler from about 550K to about 400K bytes" (Lowry and Medlock [1969]).

A compiler for revised Pascal was written in old Pascal and translated into P₆₀₀₀‡6000. As in Example 11.2, the symbol ‡ marks a source of inefficiency. The old compiler did not generate sufficiently efficient code. “Therefore, the compiler speed of [P₆₀₀₀‡6000] was rather moderate and its storage requirements quite high (Ammann [1981]).” Revisions to Pascal were small enough that the compiler P₀6000 could be hand-translated with little effort into Pp6000 and run through the inefficient compiler P₆₀₀₀‡6000 to obtain a clean implementation. □

11.3 THE COMPILER-DEVELOPMENT ENVIRONMENT

In a real sense, a compiler is just a program. The environment in which this program is developed can affect how quickly and reliably the compiler is implemented. The language in which the compiler is implemented is equally as important. Although compilers have been written in languages like Fortran, a clear choice for most compiler writers is a systems-oriented language like C.

If the source language itself is a new systems-oriented language, then it makes good sense to write the compiler in its own language. Using the bootstrapping techniques discussed in the previous section, compiling the compiler helps debug the compiler.

The software-construction tools in the programming environment can greatly facilitate the creation of an efficient compiler. In writing a compiler, it is customary to partition the overall program into modules, where each module may be processed in quite different ways. A program that manages the processing of these modules is an indispensable aid to the compiler writer. The UNIX system contains a command called *make* (Feldman [1979a]) that manages and maintains the modules making up a computer program; *make* keeps track of the relationships between the modules of the program, and issues only those commands needed to make the modules consistent after changes are made.

Example 11.4. The command *make* reads the specification of what tasks need to be done from a file called *makefile*. In Section 2.9, we constructed a translator by compiling seven files with a C compiler, each depending on a global header file *global.h*. To show how the task of putting the compiler together can be done by *make*, suppose we call the resulting compiler *trans*. The *makefile* specification might look like:

```
OBJS = lexer.o parser.o emitter.o symbol.o\
      init.o error.o main.o

trans: $(OBJS)
      cc $(OBJS) -o trans

lexer.o parser.o emitter.o symbol.o\
      init.o error.o main.o: global.h
```

The equal sign on the first line makes `OBJS` on the left stand for the seven object files on the right. (Long lines can be split by placing a backslash at the end of the continued portion.) The colon on the second line says `trans` on its left depends on all the files in `OBJS`. Such a dependency line can be followed by a command to “make” the file to the left of the colon. The third line therefore says that the target program `trans` is created by linking the object files `lexer.o`, `parser.o`, . . . , `main.o`. However, *make* knows that it must first create the object files; it automatically does this by looking for the corresponding source files `lexer.c`, `parser.c`, . . . , `main.c`, and compiling each with the C compiler to create the corresponding object files. The last line of *makefile* says that all seven object files depend on the global header file `global.h`.

The translator is created by just typing the command *make*, which causes the following commands to be issued:

```
cc -c lexer.c
cc -c parser.c
cc -c emitter.c
cc -c symbol.c
cc -c init.c
cc -c error.c
cc -c main.c
cc lexer.o parser.o emitter.o symbol.o\
    init.o error.o main.o -o trans
```

Subsequently, a compilation will be redone only if a dependent source file is changed after the last compilation. Kernighan and Pike [1984] contains examples of the use of *make* to facilitate the construction of a compiler. □

A profiler is another useful compiler-writing tool. Once the compiler is written, a profiler can be used to determine where the compiler spending its time as it compiles a source program. Identification and modification of the hot spots in the compiler can speed up the compiler by a factor of two or three.

In addition to software-development tools, a number of tools have been developed specifically for the compiler-development process. In Section 3.5, we described the generator Lex that can be used to automatically produce a lexical analyzer from a regular-expression specification of a lexical analyzer; in Section 4.9, we described a generator Yacc that can be used to automatically produce an LR parser from a grammatical description of the syntax of the language. The *make* command described above will automatically invoke Lex and Yacc wherever needed. In addition to lexical-analyzer generators and parser generators, attribute-grammar generators and code-generator generators have been created to help build compiler components. Many of these compiler-construction tools have the desirable property that they will catch bugs in the specification of the compiler.

There has been some debate on the efficiency and convenience of program

generators in compiler construction (Waite and Carter [1985]). The observed fact is that well-implemented program generators are a significant aid in producing reliable compiler components. It is much easier to produce a correct parser using a grammatical description of the language and a parser generator, than implementing a parser directly by hand. An important issue, however, is how well these generators interface with one another and with other programs. A common mistake in the design of a generator is to assume that it is the center of the design. A better design has the generator produce subroutines with clean interfaces that can be called by other programs (Johnson and Lesk [1978]).

11.4 TESTING AND MAINTENANCE

A compiler must generate correct code. Ideally, we would like to have a computer mechanically verify that a compiler faithfully implements its specification. Several papers do discuss the correctness of various compiling algorithms but unfortunately, compilers are rarely specified in such a manner that an arbitrary implementation can be mechanically checked against a formal specification. Since compilers are usually rather complex functions, there is also the issue of verifying that the specification itself is correct.

In practice, we must resort to some systematic method of testing the compiler in order to increase our confidence that it will work satisfactorily in the field. One approach used successfully by many compiler writers is the "regression" test. Here we maintain a suite of test programs, and whenever the compiler is modified, the test programs are compiled using both the new and the old versions of the compiler. Any differences in the target programs produced by the two compilers are reported to the compiler writer. The UNIX system command *make* can also be used to automate the testing.

Choosing the programs to include in the test suite is a difficult problem. As a goal, we would like the test programs to exercise every statement in the compiler at least once. It usually requires great ingenuity to find such a test suite. Exhaustive test suites have been constructed for several languages (Fortran, TEX, C, etc.). Many compiler writers add to the regression tests programs that have exposed bugs in previous versions of their compiler; it is frustrating to have an old bug reappear because of a new correction.

Performance testing is also important. Some compiler writers check that new versions of the compiler generate code that is approximately as good as the previous version by doing timing studies as part of the regression test.

Maintenance of a compiler is another important problem, particularly if the compiler is going to be run in different environments, or if people involved in the compiler project come and go. A crucial element in being able to maintain a compiler is good programming style and good documentation. The authors know of one compiler that was written using only seven comments, one of which read "This code is cursed." Needless to say, such a program is somewhat difficult to maintain by anyone except perhaps the original writer.

Knuth [1984b] has developed a system called WEB that addresses the problem of documenting large programs written in Pascal. WEB facilitates literate programming; the documentation is developed at the same time as the code, not as an afterthought. Many of the ideas in WEB can be applied equally well to other languages.

A Look at Some Compilers

This chapter discusses the structure of some existing compilers for a text-formatting language, Pascal, C, Fortran, Bliss, and Modula 2. Our intent is not to advocate the designs presented here to the exclusion of others, but rather to illustrate the variety that is possible in the implementation of a compiler.

The compilers for Pascal were chosen because they influenced the design of the language itself. The compilers for C were chosen because C is the primary programming language on the UNIX operating system. The Fortran H compiler was chosen because it has significantly influenced the development of optimization techniques. BLISS/11 was chosen to illustrate the design of a compiler whose goal is to optimize space. The DEC Modula 2 compiler was chosen because it uses relatively simple techniques to produce excellent code, and was written by one person in a few months.

12.1, EQN, A PREPROCESSOR FOR TYPESETTING MATHEMATICS

The set of possible inputs to a number of computer programs can be viewed as a little language. The structure of the set can be described by a grammar, and syntax-directed translation can be used to precisely specify what the program does. Compiler technology can then be applied to implement the program.

One of the first compilers for little languages in the UNIX programming environment was EQN by Kernighan and Cherry [1975]. As described briefly in Section 1.2, EQN takes input like “ E sub 1” and generates commands for the text formatter TROFF to produce output of the form “ E_1 ”.

The implementation of EQN is sketched in Fig. 12.1. Macro preprocessing (see Section 1.4) and lexical analysis are done together. The token stream after lexical analysis is translated during parsing into text-formatting commands. The translator is constructed using the parser generator Yacc, described in Section 4.9.

The approach of treating the input to EQN as a language and applying compiler technology to construct a translator has several benefits noted by the authors.

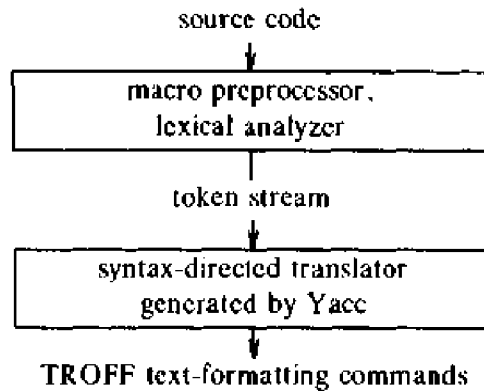


Fig. 12.1. EQN implementation.

1. *Ease of implementation.* "Construction of a working system sufficient to try out significant examples required perhaps a person-month."
2. *Language evolution.* A syntax-directed definition facilitates changes in the input language. Over the years EQN has evolved in response to user needs.

The authors conclude by observing that "defining a language, and building a compiler for it using a compiler/compiler seems like the only sensible way to do business."

12.2 COMPILERS FOR PASCAL

The design of Pascal and the development of the first compiler for it "were interdependent," as Wirth [1971] notes. It is therefore instructive to examine the structure of the compilers for the language written by Wirth and his colleagues. The first (Wirth [1971]) and the second compilers (Ammann [1981, 1977]) generated absolute machine code for the CDC 6000 series machines. Portability experiments with the second compiler led to the Pascal-P compiler that generates code, called P-code, for an abstract stack machine (Nori et al. [1981]).

Each of the above compilers is a one-pass compiler organized around a recursive-descent parser, like the "baby" front end in Chapter 2. Wirth [1971] observes that "it turned out to be relatively easy to mould the language according to [the restrictions of the parsing method]." The organization of the Pascal-P compiler is shown in Fig. 12.2.

The basic operations of the abstract stack machine used by the Pascal-P compiler reflect the needs of Pascal. Storage for the machine is organized into four areas:

1. code for the procedures,
2. constants,

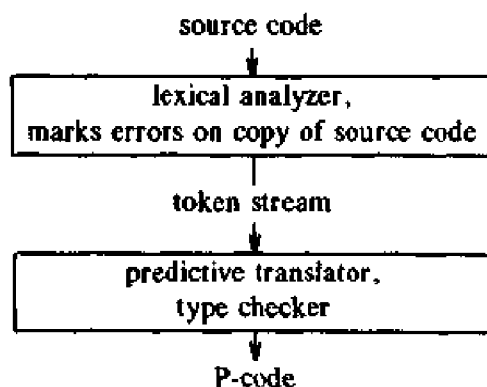


Fig. 12.2. Pascal-P compiler.

3. a stack for activation records, and
4. a heap for data allocated by applying the `new` operator.¹

Since procedures may be nested in Pascal, the activation record for a procedure contains both access and control links. A procedure call is translated into a “mark stack” instruction for the abstract machine, with the access and control links as parameters. The code for a procedure refers to the storage for a local name using a displacement from an end of the activation record. Storage for nonlocals is referred to by a pair, consisting of the number of access links to be traversed and a displacement, as in Section 7.4. The first compiler used a display for efficient access to nonlocals.

Ammann [1981] draws the following conclusions from the experience with writing the second compiler. On the one hand, the one-pass compiler was easy to implement and generated modest input/output activity (the code for a procedure body is compiled in memory and written out as a unit to secondary storage). On the other hand, the one-pass organization “imposes severe restrictions on the quality of the generated code and suffers from relatively high storage requirements.”

12.3 THE C COMPILERS

C is a general-purpose programming language designed by D. M. Ritchie and is used as the primary programming language on the UNIX operating system (Ritchie and Thompson [1974]). UNIX itself is written in C and has been moved to a number of machines, ranging from micro-processors to large mainframes, by first moving a C compiler. This section briefly describes the overall structure of the compiler for the PDP-11 by Ritchie [1979] and the PCC family of portable C compilers by Johnson [1979]. Three-quarters of the

¹ Bootstrapping is facilitated by the fact that the compiler, written in the subset it compiles, uses the heap like a stack, so a simple heap manager can be used initially.

code in PCC is independent of the target machine. All these compilers are essentially two-pass; the PDP-11 compiler has an optional third pass that does optimization on the assembly-language output, as indicated in Fig. 12.3. This peephole optimization phase eliminates redundant or inaccessible statements.

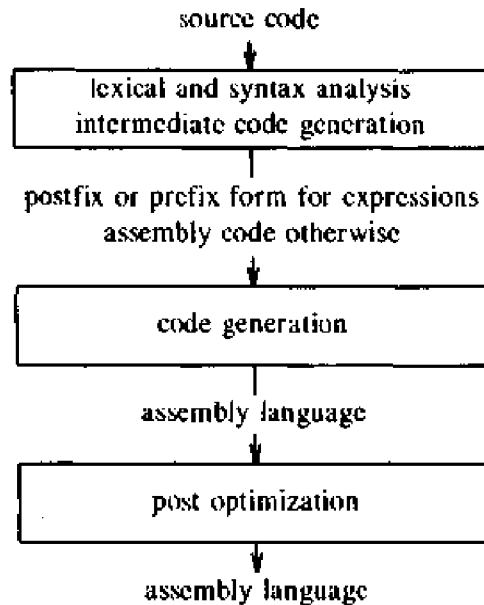


Fig. 12.3. Pass structure of C compilers.

Pass I of each compiler does lexical analysis, syntax analysis, and intermediate code generation. The PDP-11 compiler uses recursive descent to parse everything except expressions, for which operator precedence is used. The intermediate code consists of postfix notation for expressions and assembly code for control-flow statements. PCC uses an LALR(1) parser generated by Yacc. Its intermediate code consists of prefix notation for expressions and assembly code for other constructs. In each case, storage allocation for local names is done during the first pass, so these names can be referred to using offsets into an activation record.

Within the back end, expressions are represented by syntax trees. In the PDP-11 compiler, code generation is implemented by a tree walk, using a strategy similar to the labeling algorithm in Section 9.10. Modifications to that algorithm have been made to assure that register pairs are available for operations that need them and to take advantage of operands that are constants.

Johnson [1978] reviews the influence of theory on PCC. In both PCC and PCC2, a subsequent version of the compiler, code for expressions is generated by tree rewriting. The code generator in PCC examines a source language statement at a time, repeatedly finding maximal subtrees that can be computed without stores, using the available registers. Labels computed as in Section

9.10 identify subexpressions to be computed and stored in temporaries. Code to evaluate and store the values represented by these subtrees is generated by the compiler as the subtrees are selected. The rewriting is more evident in PCC2, whose code generator is based on the dynamic programming algorithm of Section 9.11.

Johnson and Ritchie [1981] describe the influence of the target machine on the design of activation records and the procedure call/return sequence. The standard library function `printf` can have a variable number of arguments, so the design of the calling sequence on some machines is dominated by the need to allow variable-length argument lists.

12.4 THE FORTRAN H COMPILERS

The original Fortran H compiler written by Lowry and Medlock [1969] was an extensive and fairly powerful optimizing compiler built using methods that largely predate those described in this book. Several attempts at increased performance have been made; an "extended" version of the compiler was developed for the IBM/370, and an "enhanced" version was developed by Scarborough and Kolsky [1980]. Fortran H offers the user the choice of no optimization, register optimization only, or full optimization. A sketch of the compiler in the case that full optimization is performed appears in Fig. 12.4.

The source text is treated in four passes. The first two perform lexical and syntactic analysis, producing quadruples. The next pass incorporates code optimization and register optimization and the final pass generates object code from the quadruples and register assignments.

The lexical analysis phase is somewhat unusual, since its output is not a stream of tokens but a stream of "operator-operand pairs," which are roughly equivalent to an operand token together with the preceding nonoperand token. It should be noted that in Fortran, like most languages, we never have two consecutive operand tokens such as identifiers or constants; rather, two such tokens are always separated by at least one punctuation token.

For example, the assignment statement

$$A = B(I) + C$$

would be translated into the sequence of pairs:

"assignment statement"	A
=	B
(s	I
)	-
+	C

The lexical analysis phase distinguishes between a left parenthesis whose job is to introduce a list of parameters or subscripts from one whose job is to group operands. Thus, the symbol "(s" is intended to represent a left parenthesis used as a subscripting operator. Right parentheses never have an operand

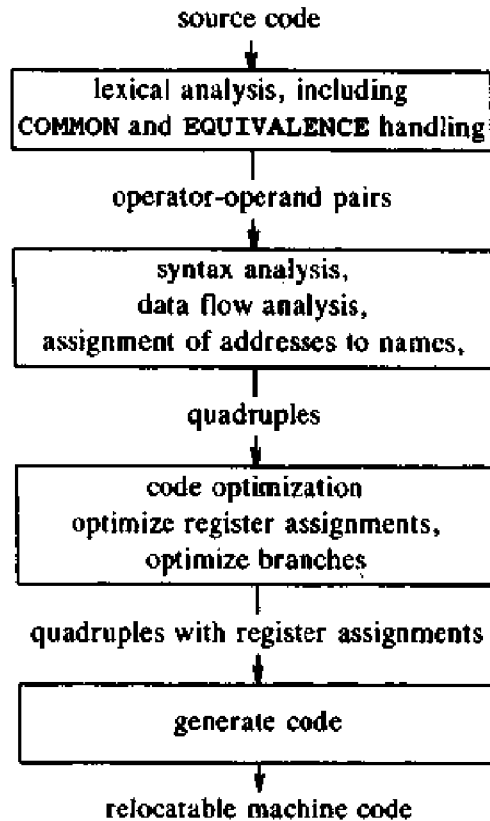


Fig. 12.4. Outline of Fortran H compiler.

following, and Fortran H does not distinguish the two roles for right parentheses.

Associated with lexical analysis is the processing of **COMMON** and **EQUIVALENCE** statements. It is possible at this stage to map out each **COMMON** block of storage, as well as the storage blocks associated with the subroutines, and to determine the location of each variable mentioned by the program in one of these static storage areas.

Since Fortran has no structured control statements like **while**-statements, parsing, except for expressions, is quite straightforward, and Fortran H simply uses an operator-precedence parser for expressions. Some very simple local optimizations are performed during the generation of quadruples; for example, multiplications by powers of 2 are replaced by left-shift operations.

Code Optimization in Fortran H

Each subroutine is partitioned into basic blocks, and the loop structure is deduced by finding flow-graph edges whose heads dominate their tails, as described in Section 10.4. The compiler performs the following optimizations.

1. *Common subexpression elimination.* The compiler looks for local common subexpressions and for expressions that are common to a block B and one or more blocks that B dominates. Other instances of common subexpressions are not detected. Further, the detection of common subexpressions is done one expression at a time, rather than using the bit-vector method described in Section 10.6. Interestingly, in developing the "enhanced" version of the compiler, the authors found that a major speedup was possible by using bit-vector methods.
2. *Code motion.* Loop-invariant statements are removed from loops essentially as described in Section 10.7.
3. *Copy propagation.* Again, this is done one copy statement at a time.
4. *Induction variable elimination.* This optimization is performed only for variables that are assigned once in the loop. Instead of using the "family" approach described in Section 10.7, multiple passes through the code are made to detect induction variables that belong to the family of some other induction variable.

Although data flow analysis is done in a one-at-a-time style, the values corresponding to what we have called *in* and *out* are stored as bit vectors. However, in the original compiler, a limit of length 127 was placed on these vectors, so large programs have only the most frequently used of their variables involved in optimizations. The enhanced version increases the limit but does not remove it.

Algebraic Optimizations

As Fortran is frequently used for numerical calculations, algebraic optimization is dangerous, since transformations of expressions can, in computer arithmetic, introduce overflows or losses of precision that are not visible if we take an idealized view of algebraic simplification. However, algebraic transformations involving integers are generally safe, and the enhanced version of the compiler does some of this optimization in the case of array references only.

In general, an array reference like $A(I, J, K)$ involves an offset calculation in which an expression of the form $aI + bJ + cK + d$ is computed; the exact values of the constants depend on the location of A and the dimensions of the array. If, say, I and K were constants, either numerical constants or loop-invariant variables, then the compiler applies the commutative and associative law to get an expression $bJ + e$, where $e = aI + cK + d$.

Register Optimization

Fortran H divides registers into three classes. These sets of registers are used for local register optimization, global register optimization, and "branch optimization." The exact number of registers in each class can be adjusted by the compiler, within limits.

Global registers are allocated on a loop-by-loop basis to the most frequently referenced variables in that loop. A variable that qualifies for a register in one loop L , but not in the loop immediately containing L , is loaded in the pre-header of L and stored on exit from L .

Local registers are used within a basic block to hold the results of one statement until it is used in a subsequent statement or statements. Only if not enough local registers exist is a temporary value stored. The compiler tries to compute new values in the register holding one of its operands, if that operand is subsequently dead. In the enhanced version, an attempt is made to recognize the situation where global registers may be exchanged with other registers to increase the number of times that an operation can take place in the register holding one of its operands.

Branch optimization is an artifact of the IBM/370 instruction set, which puts a significant premium on jumping only to locations that can be expressed as the contents of some register, plus a constant in the range 0 to 4095. Thus, Fortran H allocates some registers to hold addresses in the code space, at intervals of 4096 bytes, to allow efficient jumps in all but extremely large programs.

12.5 THE BLISS/11 COMPILER

This compiler implements the systems programming language Bliss on a PDP-11 (Wulf et al. [1975]). In a sense, it is an optimizing compiler from a world that has ceased to exist, a world where memory space was at enough of a premium that it made sense to perform optimizations whose sole purpose was to reduce space rather than time. However, most of the optimizations performed by the compiler save time as well, and descendants of this compiler are in use today.

The compiler is worth our attention for several reasons. Its optimization performance is strong, and it performs a number of transformations found almost nowhere else. Further, it pioneered the "syntax-directed" approach to optimization, as discussed in Section 10.5. That is, the language Bliss was designed to produce only reducible flow graphs (it has no goto's). Thus, it was possible for data flow analysis to be performed on the parse tree directly, rather than on a flow graph.

The compiler operates in a single pass, with one procedure completely processed before the next is read in. The designers view the compiler as composed of five modules, as shown in Fig. 12.5.

LEXSYNFLO performs lexical analysis and parsing. A recursive-descent parser is used. As BLISS permits no goto-statements, all flow graphs of BLISS procedures are reducible. In fact, the syntax of the language enables us to build the flow graph, and determine loops and loop entries, as we parse. LEXSYNFLO does so, and also determines common subexpressions and a variant of ud- and du-chains, taking advantage of the structure of reducible flow graphs. Another important job of LEXSYNFLO is to detect groups of

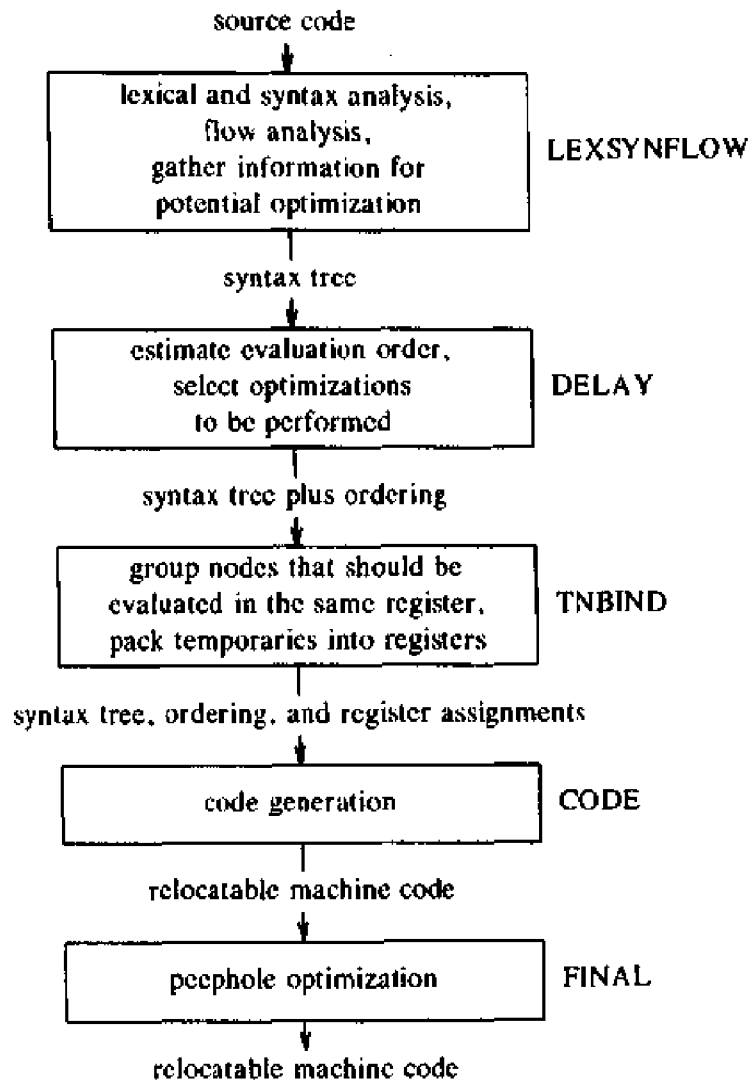


Fig. 12.5. The BLISS/11 compiler.

similar expressions. These are candidates for replacement by a single subroutine. Note that this replacement makes the program run more slowly but can save space.

The module DELAY examines the syntax tree to determine which particular instances of the usual optimizations, such as invariant code motion and elimination of common subexpressions, are actually likely to produce a profit. The order of expression evaluation is determined at this time, based on the labeling strategy of Section 9.10, modified to take into account registers that are unavailable because they are used to preserve the values of common subexpressions. Algebraic laws are used to determine whether reordering of computations should be done. Conditional expressions are evaluated either numerically or by control flow, as discussed in Section 8.4, and DELAY decides which mode is cheaper in each instance.

TNBIND considers which temporary names should be bound to registers. Both registers and memory locations are allocated. The strategy used is to first group nodes of the syntax tree that should be assigned the same register. As discussed in Section 9.6, there is an advantage to evaluating a node in the same register as one of its parents. Next, the advantage to be gained by keeping a temporary in a register is estimated by a calculation favoring those that are used several times over a short span. Registers are then assigned until used up, packing the most advantageous nodes into registers first. CODE converts the tree, with its ordering and register assignment information, to relocatable machine code.

This code is then repeatedly examined by FINAL which performs peephole optimization until no further improvement results. The improvements made include elimination of (conditional or unconditional) jumps to jumps and complementation of conditionals, as discussed in Section 9.9.

Redundant or unreachable instructions are eliminated (these could have resulted from other FINAL optimizations). Merging of similar code sequences on the two paths of a branch is attempted, as is local propagation of constants. A number of other local optimizations, some quite machine-dependent are attempted. An important one is replacement, where possible, of jump instructions by PDP-11 "branches," which require one word but are limited in their range to 128 words.

12.6 MODULA-2 OPTIMIZING COMPILER

This compiler, described in Powell [1984], was developed with the intent of producing good code, using optimizations that provide a high payoff for little effort; the author describes his strategy as looking for the "best simple" optimizations. Such a philosophy can be difficult to carry out; without experimentation and measurement, it is hard to decide what the "best simple" optimizations are in advance, and some of the decisions made in the Modula-2 compiler are probably inappropriate for a compiler providing maximum optimization. Nevertheless, the strategy did achieve the author's goal of producing excellent code with a compiler that was written in a few months by one person. The five passes of the front end of the compiler are sketched in Fig. 12.6.

The parser was generated using Yacc, and it produces syntax trees in two passes, since Modula variables do not have to be declared before use. An attempt to make this compiler compatible with existing facilities was made. The intermediate code is P-code for compatibility with many Pascal compilers. The procedure call format for this compiler agrees with that of the Pascal and C compilers running under Berkeley UNIX, so procedures written in the three languages can be integrated easily.

The compiler does not do data-flow analysis. Rather, Modula-2, like Bliss, is a language that can produce only reducible flow graphs, so the methodology